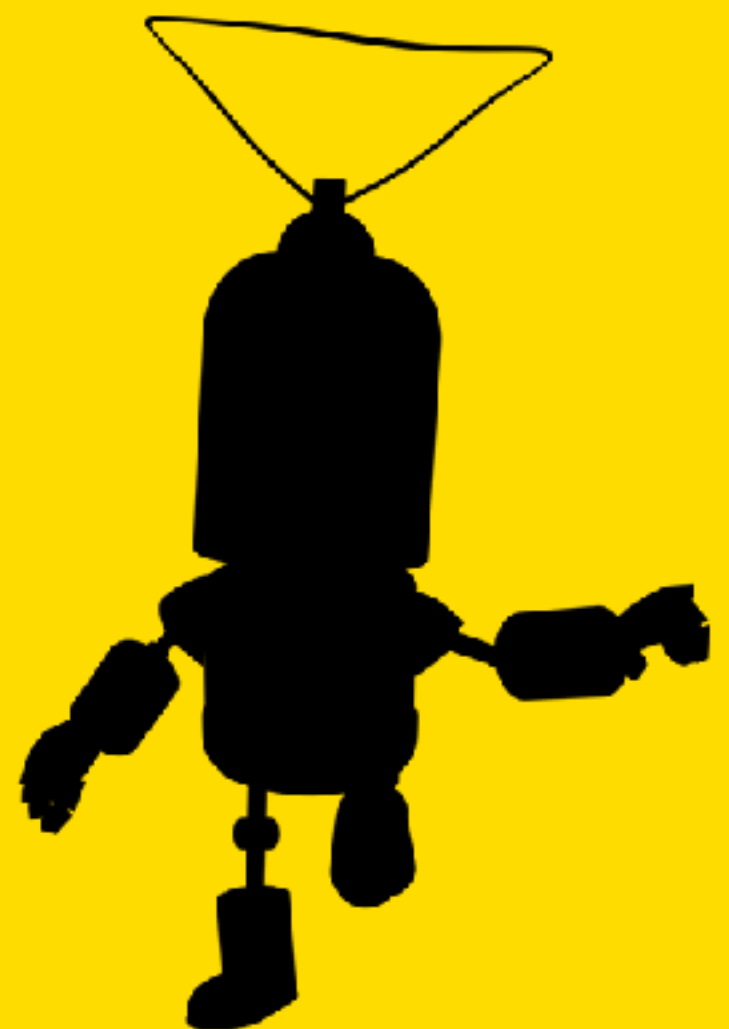


Planning

COMP9414 Artificial Intelligence

Oliver Obst



Overview

→	1	5/6	Introduction. Agents & search		
	2	12/6	Planning		release assignment #1
	3	19/6	Knowledge representation		
	4	26/6	Neural networks		
	5	3/7	Reinforcement learning		deadline assignment #1 (6 Jul)
	6	10/7			release assignment #2
	7	17/7	Reasoning with uncertainty		
	8	24/7	Computer vision		
	9	31/7	Foundation models		deadline assignment #2 (31 Jul)
	10	7/8	Human-aligned intelligent robotics		

Week 2 content

- Why path search is not enough
- planning problems: initial state, goal, actions
- preconditions and effects
- STRIPS-style action representation
- forward state-space planning
- example: office delivery robot
- limits of planning and the need for explicit knowledge representation

From search to planning

Slides material and acknowledgement:

These slides contain material from

- Artificial Intelligence: A modern approach, by Peter Norvig and Stuart Russell (chapter 11)
- U Pittsburgh “Planning and Search / Classical Planning”, by Jan Wiebe
- Automated Planning: Theory and Practice, Dana Nau (chapter 2 and chapter 4)
- John Hopkins University, Artificial Intelligence; Planning slides by Philipp Koehn
- the ai9414 python package

Today

Core ideas

- Why path search is not enough for many tasks
- Planning problems: initial state, goal, actions
- Preconditions, effects, and STRIPS-style action models
- Forward state-space planning

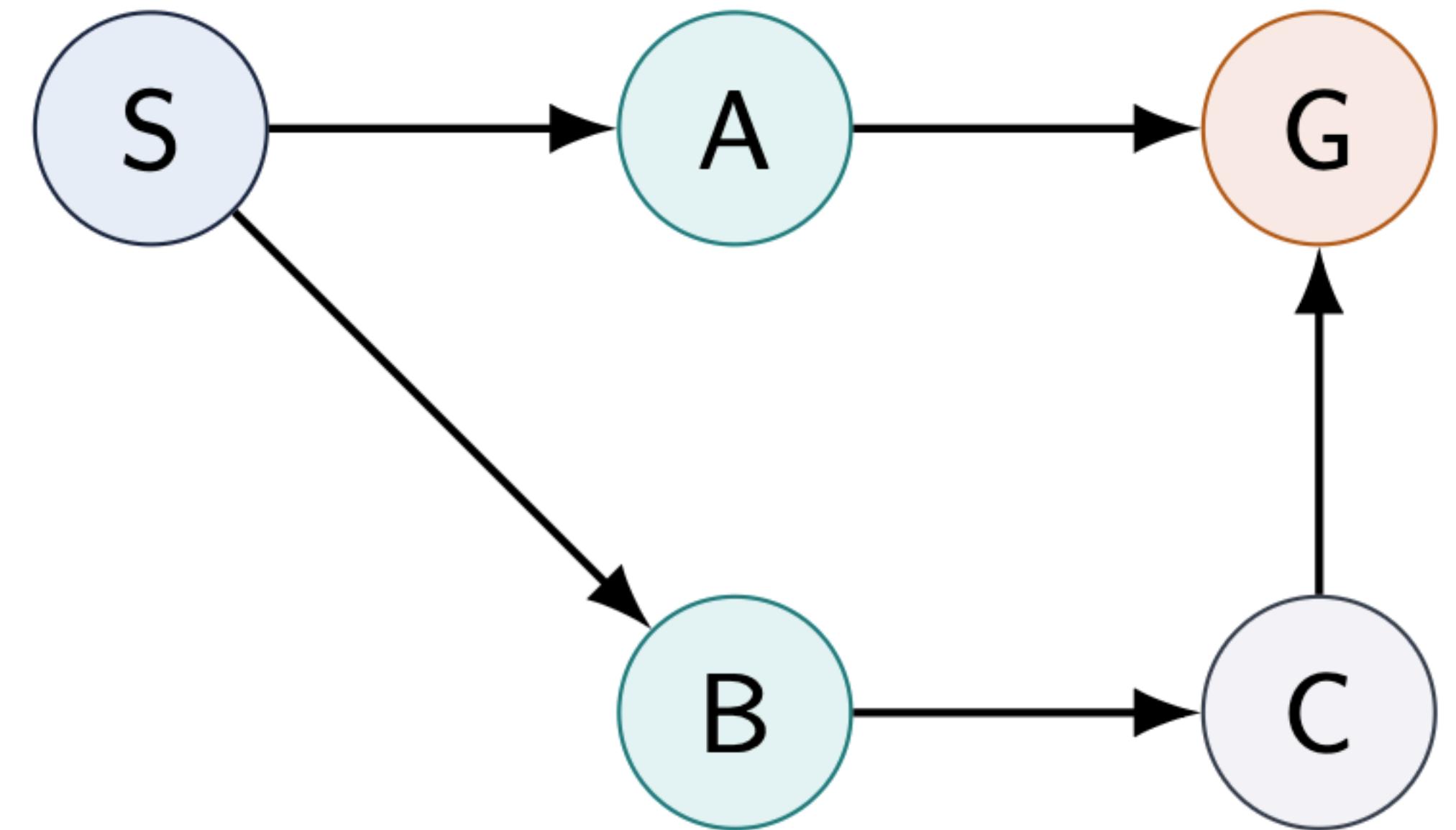
Running example

- Office delivery robot
- Parcel, keycard, locked rooms
- How a plan follows from facts about the world, not from a drawn path
- Why explicit knowledge representation matters

Path search has been useful

Path-search idea

- A state is whatever the programmer puts in a node.
- Successors are generated by code.
- The goal test is code.
- The answer is a path from the start node to a goal node.



This is enough when the graph is already the right abstraction.

Planning tasks are not just routes

Task

Deliver a parcel to a locked lab

The robot may need to:

- Get a keycard
- Unlock a door
- Pick up and drop the parcel
- Avoid attempting actions whose preconditions are false

Why a route is not enough

- A shortest path could cross a locked door.
- Picking up a parcel may prevent picking up something else.
- May need to choose one action before another one.
- There may be many possible (legal) actions, but also many irrelevant ones.



Go to wooclap.com

Enter the event code in the top banner

Event code

CBADXF



What's missing?

Suppose a route planner proposes:

corridor → mail_room → corridor → lab

What is the most important missing information?

- a. The geometric distance between rooms.
- b. Whether each intended action is legal when attempted.
- c. The alphabetical order of the room names.
- d. The number of alternative routes through office B.

What's missing: action legality

Planning is not just choosing adjacent rooms.
It needs to know whether the robot can:

- enter the lab while the door is locked;
- carry the parcel and still use the keycard;
- make the goal fact `at(parcel, lab)` true.

This is why the action model matters.

Search vs planning

	Search	Planning
States	Data structures chosen by the program	Sets of logical facts about the world
Actions	Successor code or graph edges	Preconditions and outcomes
Goal	Goal-test code	A condition, usually a conjunction of facts
Plan	Sequence from the start node	Sequence of applicable domain actions

Planning still uses search, but it opens up the representation of actions and goals so the planner can reason about relevance.

In the office robot domain, a room route is only useful if each move, pickup, unlock, and drop action is applicable when it is attempted.

Assumptions in planning

A clean model

- Finite set of possible facts
- Fully observable state
- Deterministic actions
- The world is static while planning

The plan model

- Attainment goals
- Sequential actions
- Implicit time
- Offline planning before execution

These assumptions are restrictive, but they let us define planning precisely.

Planning domain

Domain knowledge

- fact templates such as $\text{at}(x, y)$ and domain objects
- A collection of operators $\mathcal{O}$
- Static facts such as room connectivity
- Action models for changing facts

Generated transition system

- States $\mathcal{S}$: sets of concrete facts, like $\text{at}(\text{robot}, \text{corridor})$
- Actions $\mathcal{A}$: concrete actions, made by filling operator variables with objects
- Transition function $\gamma(s, a)$ determined by effects

We do not list the whole graph. The domain model generates states as needed.

Logic term for later: concrete facts or actions are called “ground” – with no variables left.

What is a planning problem?

$$P = (\mathcal{O}, s_0, g)$$

Operators $\mathcal{O}$

The action schemas available in the domain.

Initial state s_0

The set of facts that are true before acting

Goal g

The facts that must be true after acting

A plan is a sequence of concrete actions, each made from an operator by choosing actual objects, that is executable from s_0 and reaches a state satisfying g .

States as sets of facts

A state

- at(robot, corridor)
- locked(lab_door)
- handempty(robot)

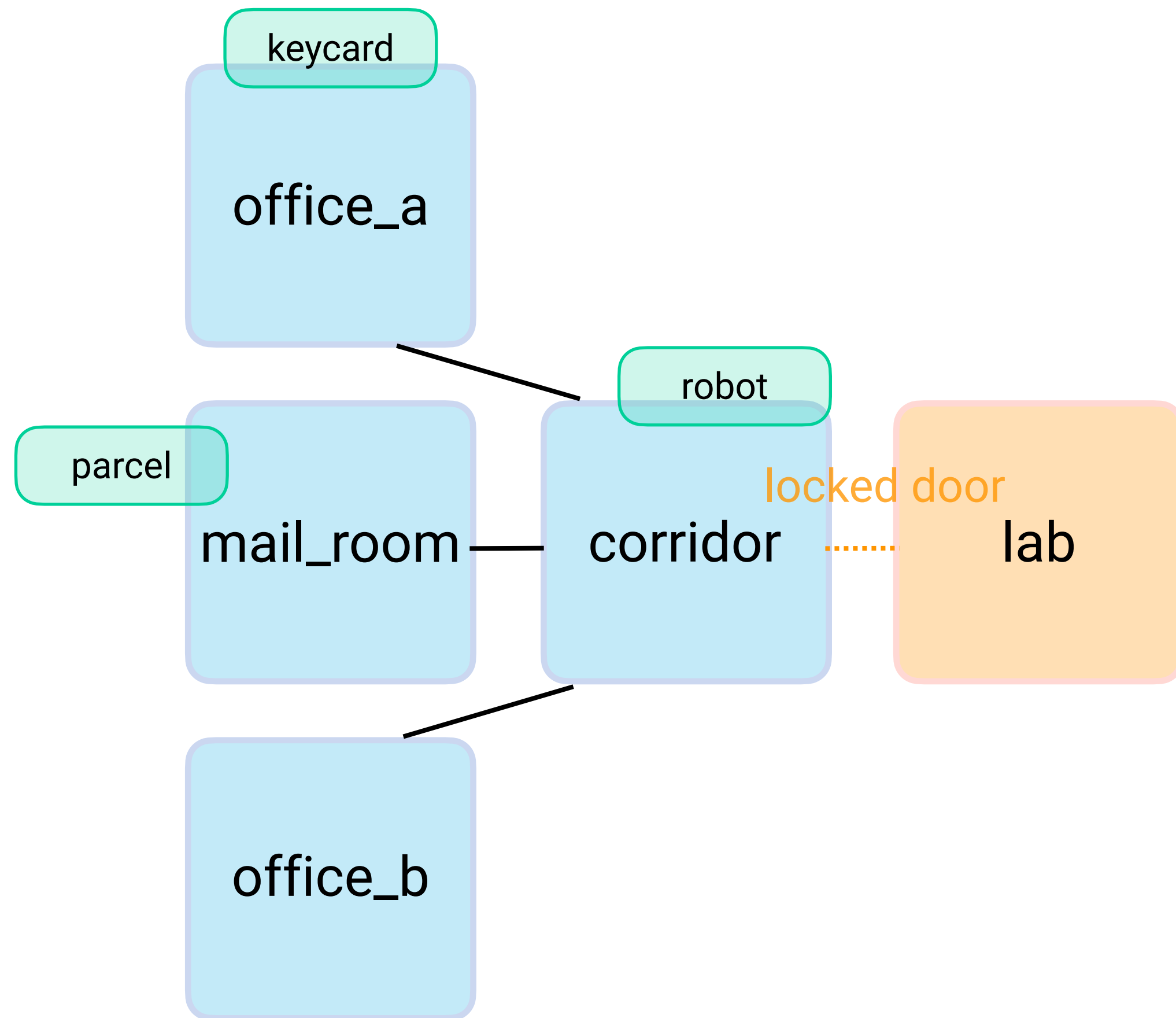
Read a fact as a true/false statement about named objects.

In logic (next week), we call facts with no variables *ground atoms*.

Closed-world convention

- Facts listed in the state are true.
- Facts not listed are treated as false.
- Static facts, such as room connected, can be included too.

The office delivery domain



Objects

- Rooms: corridor, mail room, offices, lab
- robot
- keycard
- lab door

The goal is

`at(parcel, lab)`

not just “go to the lab”.

Initial state and goal

Initial facts

- at(robot, corridor)
- at(parcel, mail_room)
- at(keycard, office_a)
- locked(lab_door)
- handempty(robot)
- door_between(lab_door, corridor, lab)
- connected(corridor, mail_room),
connected(mail_room, corridor), ...

Goal

$$G = \{ \text{at}(\text{parcel}, \text{lab}) \}$$

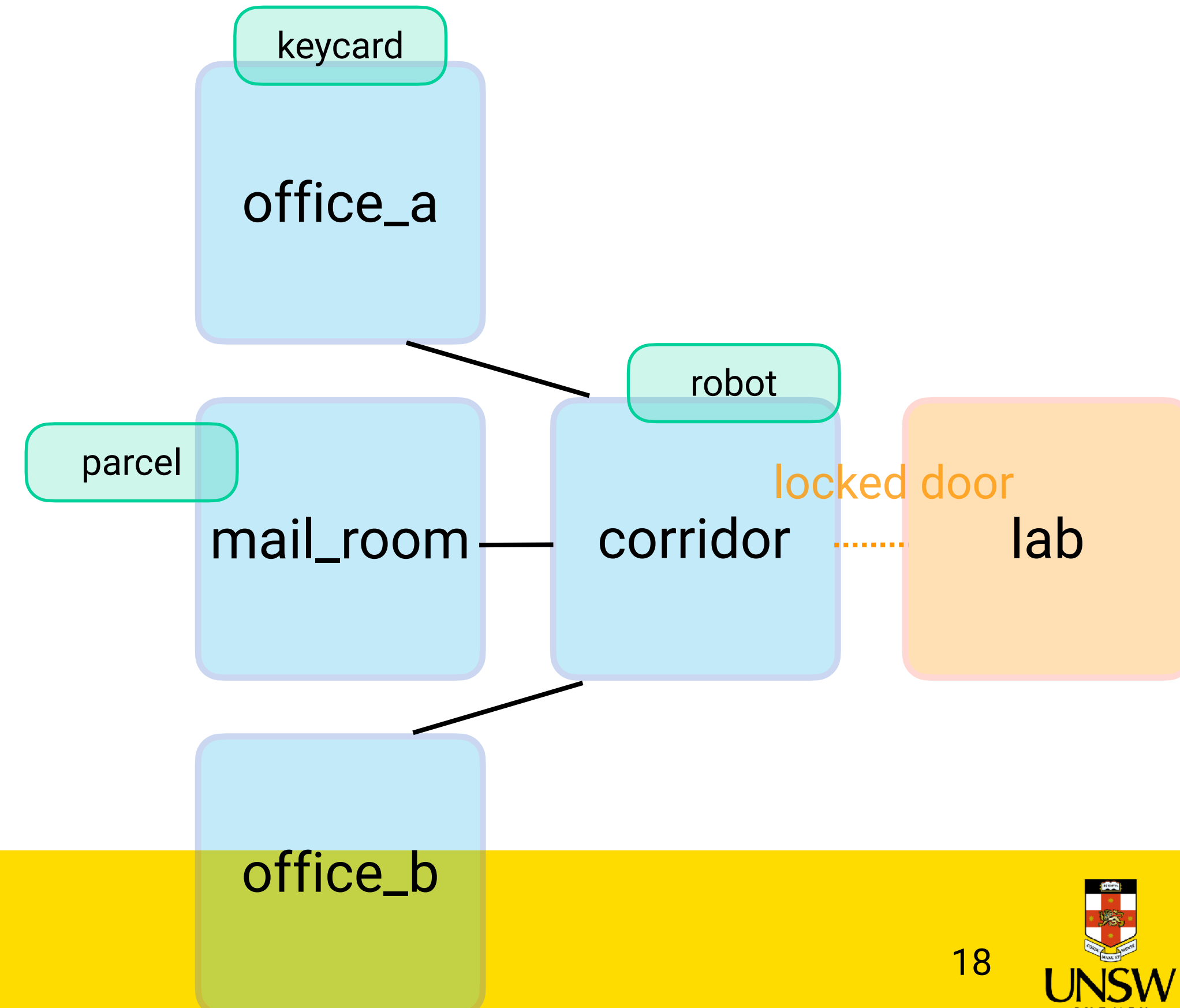
We do not have to specify every final fact. The goal only says what *must* be true at the end.

Applicable actions

In the initial state, the robot is in the corridor, the door is locked, the keycard is in office A, and the parcel is in the mail room.

Which action is applicable immediately?

- A. `move(robot, corridor, lab)`
- B. `pickup(robot, keycard, office_a)`
- C. `move(robot, corridor, office_a)`
- D. `pickup(robot, parcel, mail_room)`

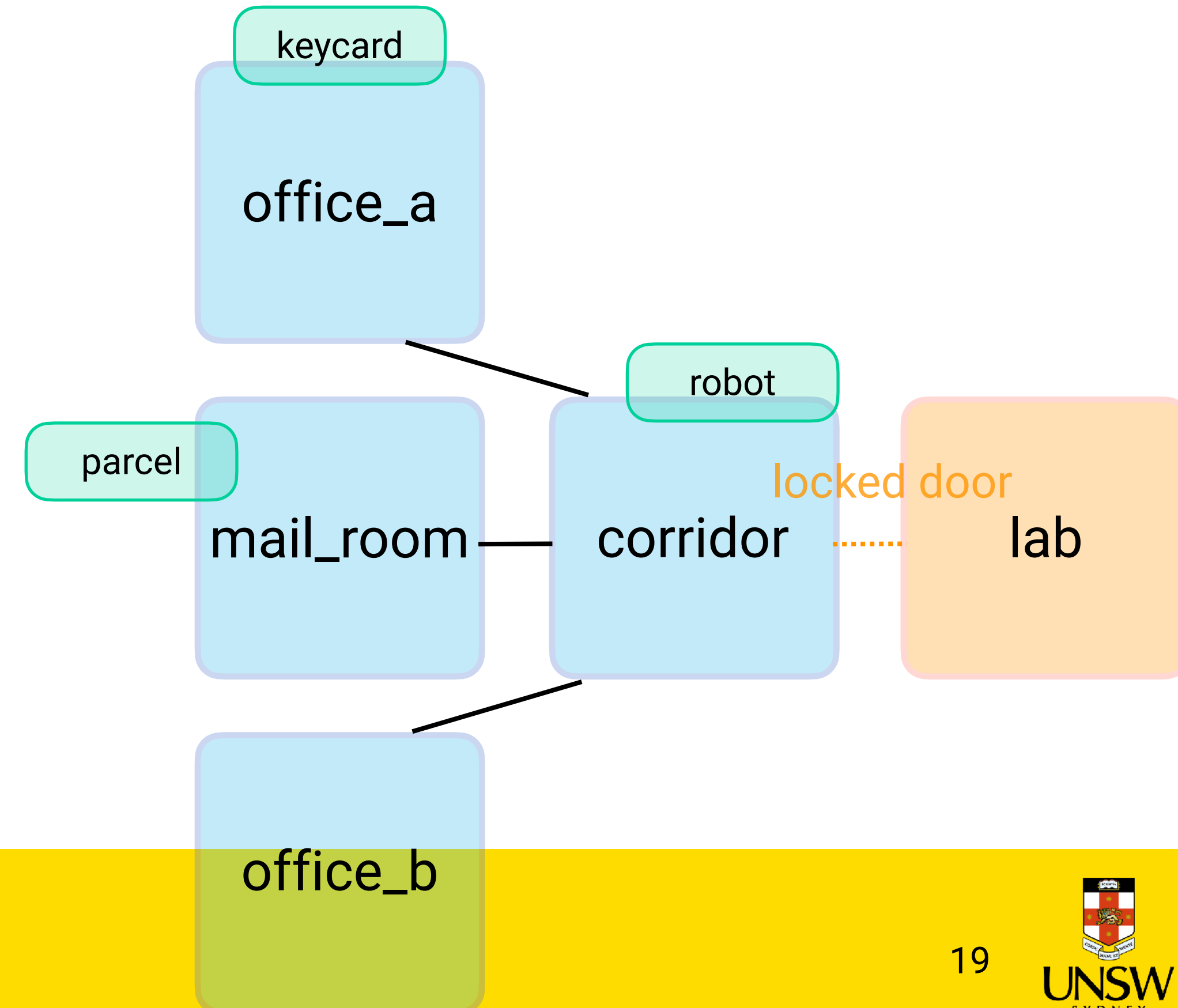


Applicable actions: correct answer C

move(robot, corridor, office_a) is applicable because:

- at(robot, corridor) is true
- connected(corridor, office_a) is true

The other actions fail because a required precondition is false.



Operators and actions

Operator schema

Contains variables:

`move(r, from, to)`

A compact description of many possible actions.

Concrete action

All variables replaced by actual objects.

`move(robot, corridor, office_a)`

This is one action the robot could try in a particular state.

Preconditions and effects

Preconditions

Facts that must already hold for the action to be legal.

If a precondition is false, the action cannot be applied.

Effects

Facts made true or false by the action.

Positive effects are added; negative effects are deleted.

The action model is the planner's causal knowledge about the world.

STRIPS - style action representation

Action

`move(robot,corridor,office_a)`

Add list

`at(robot,office_a)`

Preconditions

- `at(robot,corridor)`
- `connected(corridor,office_a)`

Delete list

`at(robot,corridor)`

STRIPS gives a tidy, restricted action language: simple preconditions and add/delete effects.

Delete effects

The robot applies:

`pickup_parcel(robot, parcel, mail_room)`

Which facts should be deleted?

- A. `at(robot, mail_room)`
- B. `at(parcel, mail_room)` and `handempty(robot)`
- C. `connected(mail_room, corridor)`
- D. `locked(lab_door)`

Delete effects

Correct answer: **B**.

After picking up the parcel:

- the parcel is no longer sitting in the mail room;
- the robot's hand is no longer empty;
- the robot is still in the mail room;
- room connectivity and the lab door lock have not changed.

PDDL-style syntax - same idea.

PDDL is a standard text format for writing planning domains and problems. It uses the same idea we have already seen: preconditions plus effects.

Action

- :action pickup_parcel
- :parameters ?robot ?parcel ?room

The same domain can be discussed as STRIPS operators, PDDL actions, or Python data structures; the semantics are preconditions plus state updates.

Semantics

- preconditions:
at(?robot,?room), at(?parcel,?room),
handempty(?robot)
- add: carrying(?robot,?parcel)
- delete:
at(?parcel,?room), handempty(?robot)

State transition rule

$$\text{Result}(s, a) = (s - \text{Del}(a)) \cup \text{Add}(a)$$

Applicable if

- $\text{Pre}(a) \subseteq s$

Then update:

- remove the delete list
- Add the add list
- All other facts persist

Example: unlock the lab door

Action

```
unlock_door(robot, keycard, lab_door,  
corridor, lab)
```

Precondition

- at(robot, corridor)
- has(robot, keycard)
- locked(lab_door)
- door_between(lab_door, corridor, lab)

Effects:

Add = {unlocked(lab_door)}

Del = {locked(lab_door)}

After this action, crossing from corridor to lab becomes applicable.

Apply the transition rule

Suppose the current state includes:

`at(robot,corridor), has(robot,keycard), locked(lab_door)`

We apply `unlock_door`. Which update is correct?

- A. `add unlocked(lab_door); delete locked(lab_door)`
- B. `add at(robot, lab); delete at(robot, corridor)`
- C. `add has(robot,keycard); delete handempty(robot)`
- D. `delete all facts not mentioned in the action.`

Frame assumption

Correct answer: **A**

STRIPS updates only the facts named in the add and delete lists.

- locked(lab_door) is removed
- unlocked(lab_door) is added
- at(robot, lab) persists
- the robot still has the keycard

More action examples

`pickup_keycard`

Pre:

`at(robot, r),
at(keycard, r),
handempty(robot)`

Add:

`has(robot, keycard)`

Del:

`at(keycard, r)`

`pickup_parcel`

Pre:

`at(robot, r),
at(parcel, r),
handempty(robot)`

Add:

`carrying(robot, keycard)`

Del:

`at(parcel, r),
handempty(robot)`

`drop_parcel`

Pre:

`at(robot, r),
carrying(robot, parcel)`

Add:

`at(parcel, r),
handempty(robot)`

Del:

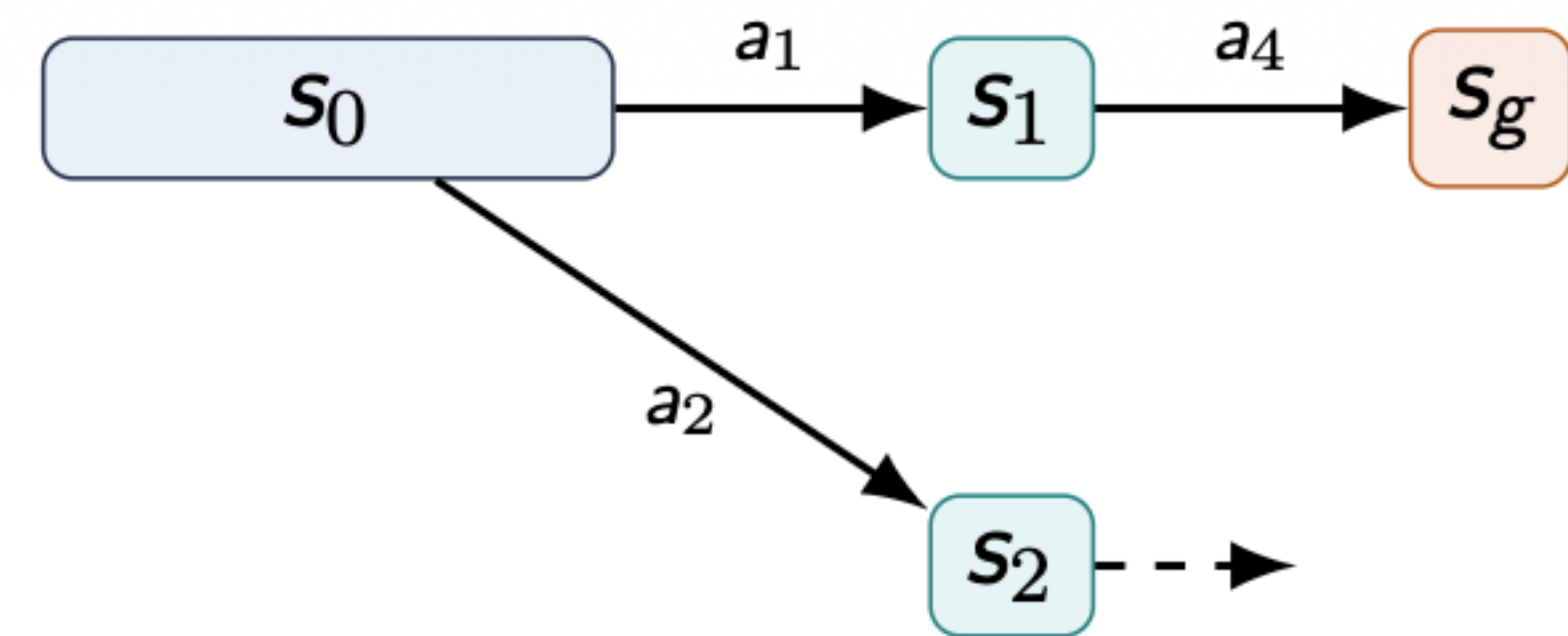
`carrying(robot, parcel)`

In this example, robot stores keycard somewhere - the parcel stays in its hand

Forward state-space planning

Idea

- Start from the initial state.
- Generate applicable actions.
- Apply each action to get successor states.
- Stop when a state satisfies the goal.



A plan is a path through the space of symbolic states.

Forward planning as BFS

1. Put I on the frontier with an empty plan.
2. Pop the next state and plan prefix.
3. If the state satisfies G, return the plan.
4. For each applicable action: add Result(s, a) with the extended plan.
5. Avoid revisiting states already seen.

BFS property:

If all actions have equal cost and the state space is finite, BFS finds a shortest action sequence.

It may still expand many irrelevant states.

First expansion in the office domain

From the initial state

$$s_0 \supset \{\text{at}(\text{robot}, \text{corridor}), \text{locked}(\text{lab_door})\}$$

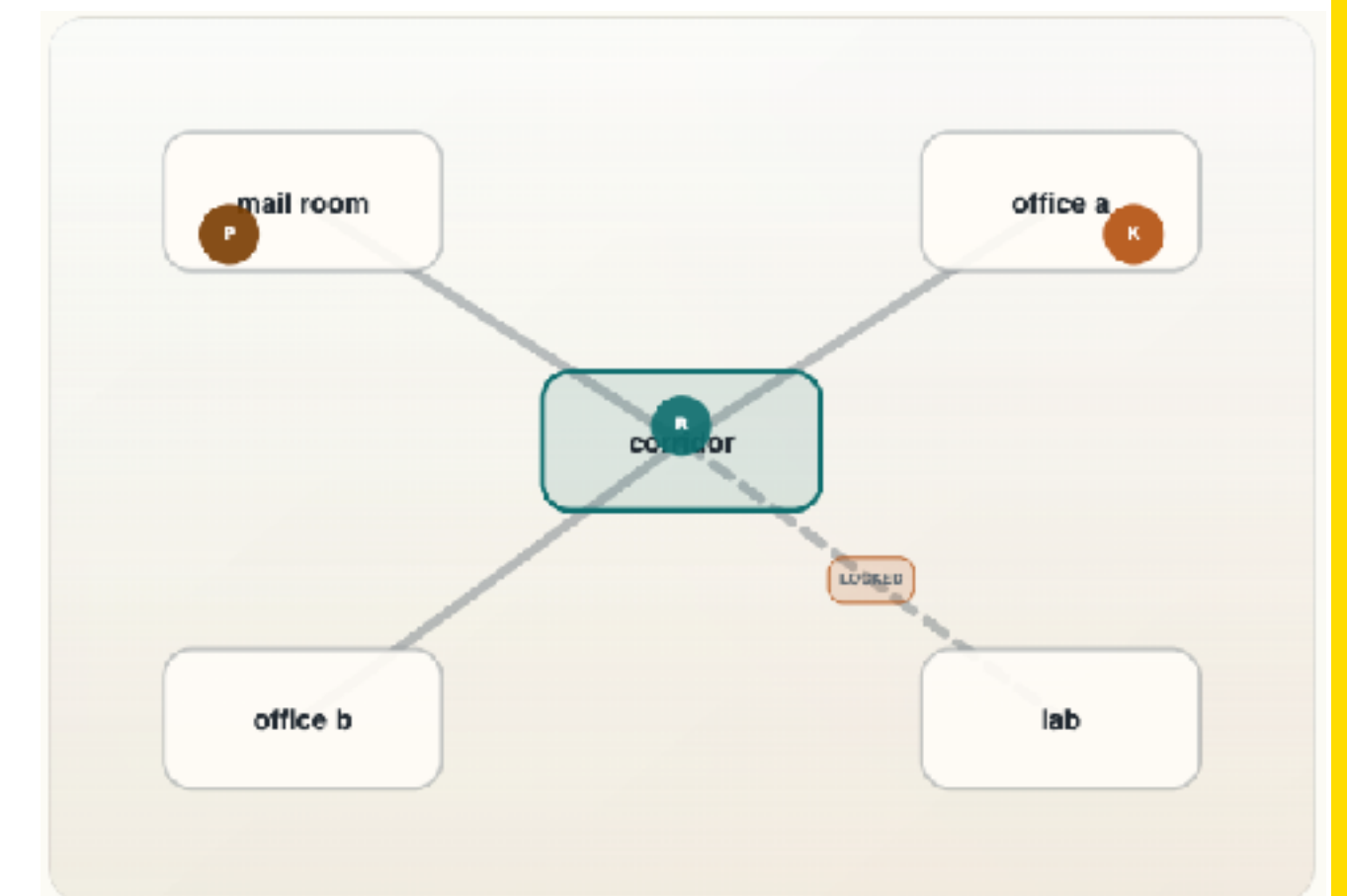
Applicable movement actions

`move(robot, corridor, office_a)`
`move(robot, corridor, mail_room)`
`move(robot, corridor, office_b)`

Not applicable yet:

`move(robot, corridor, lab)`

because `unlocked(lab_door) ∉ s0`



Next frontier

After expanding the initial state with BFS, which set best describes the next frontier?

- A. States reached by moving to office A, mail room, and office B.
- B. States reached by picking up the keycard, parcel, and lab door.
- C. Only the state reached by moving to the lab.
- D. Every possible state in the whole planning domain.

Next frontier

Correct answer: **A**

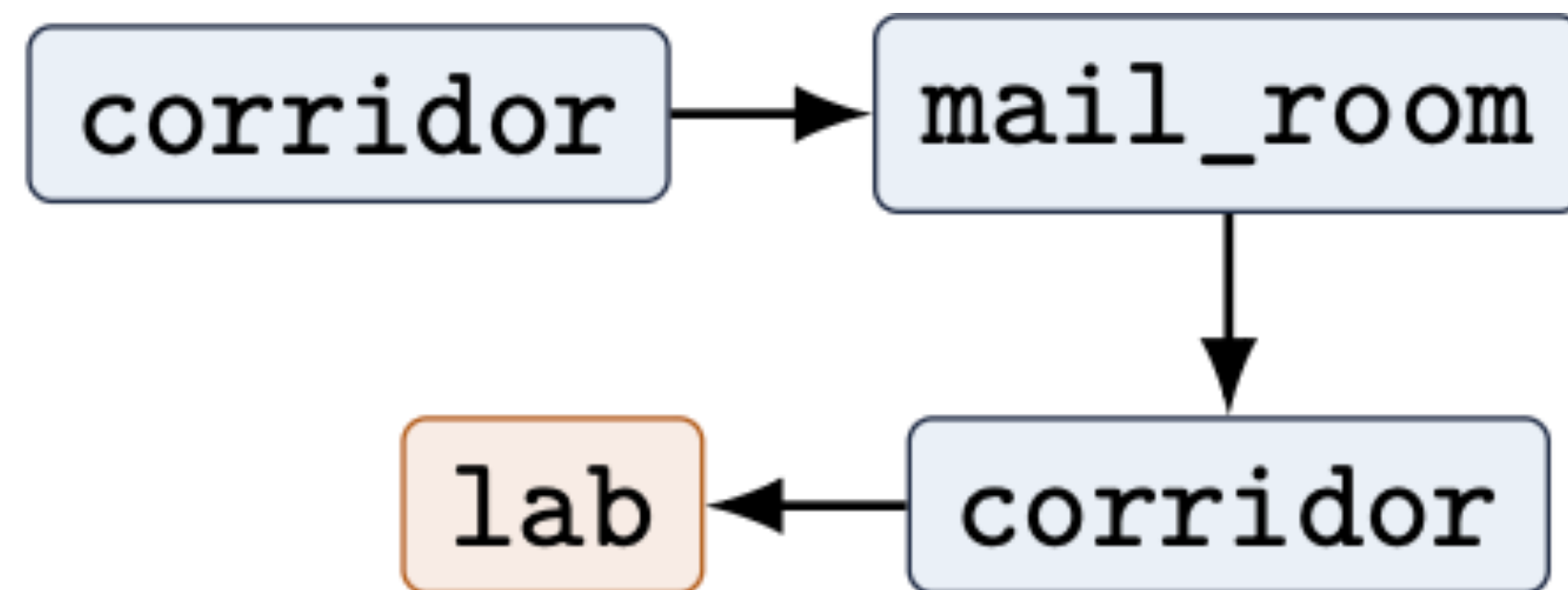
BFS expands all states at depth 1 before depth 2. From the initial state, the applicable movement actions take the robot to:

- office A;
- the mail room;
- office B.

The lab is not on the frontier yet because the locked-door precondition blocks that move.

Why naive route fails

Route idea



This looks like a path from robot to parcel to lab.

Planner's view

- The lab door is locked
- Unlocking requires the keycard
- Picking up the parcel removes `handempty(robot)`
- Getting the keycard first avoids having to put the parcel down

Found plan: Office delivery robot

- | | | | |
|-------------------|----------------------|------------------|----------------------|
| 1. move | corridor → office_a | 6. pickup_parcel | mail_room |
| 2. pickup_keycard | office_a | 7. move | mail_room → corridor |
| 3. move | office_a → corridor | 8. move | corridor → lab |
| 4. unlock_door | corridor/lab | 9. drop_parcel | lab |
| 5. move | corridor → mail_room | | |

The final state satisfies:

at(parcel, lab)

What the solver searched

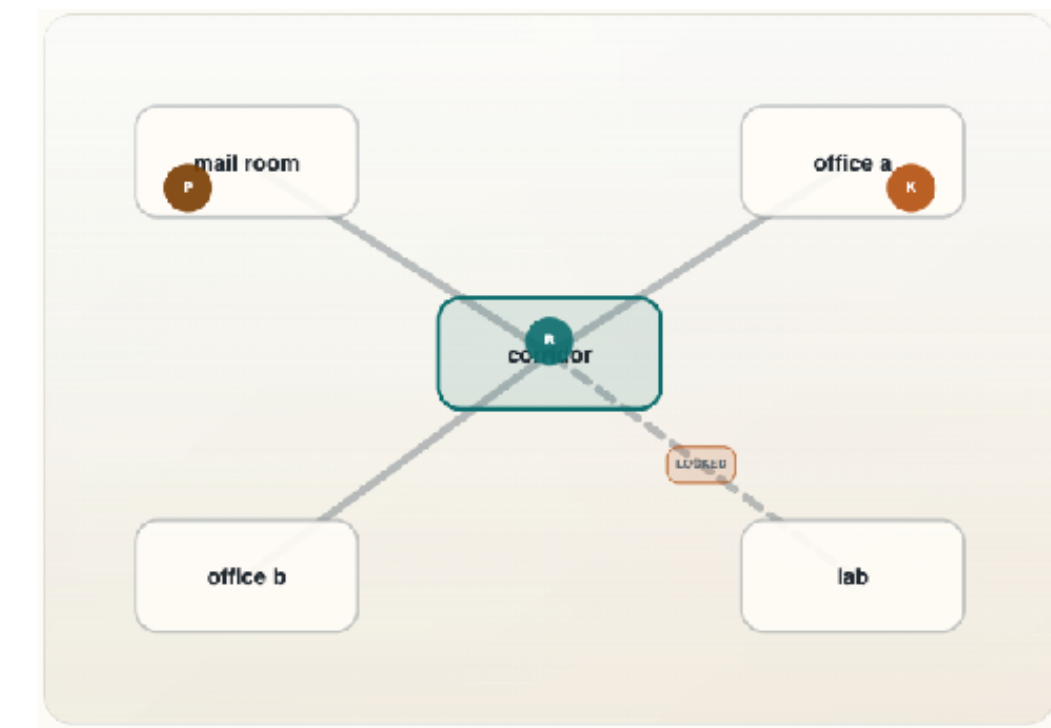
Ai9414 run:

- algorithm: forward STRIPS planning with BFS
- Plan length: 9 actions
- Expanded states: 57
- Generated states: 62
- Largest frontier: 14 states

Note. The planner is not using pixels on a map:

It searches over sets of facts (that someone has to define).

The room diagram is only a rendering of those facts for humans.



Plan trace

- | | |
|---|--------|
| <code>move(robot, corridor, office_a)</code> | STEP 1 |
| <code>pickup_keycard(robot, keycard, office_a)</code> | STEP 2 |
| <code>move(robot, office_a, corridor)</code> | STEP 3 |
| <code>unlock_door(robot, keycard, lab_door, corridor, lab)</code> | STEP 4 |
| <code>move(robot, corridor, mail_room)</code> | STEP 5 |
| <code>pickup_parcel(robot, parcel, mail_room)</code> | STEP 6 |
| <code>move(robot, mail_room, corridor)</code> | STEP 7 |
| <code>move(robot, corridor, lab)</code> | STEP 8 |
| <code>drop_parcel(robot, parcel, lab)</code> | STEP 9 |

Ai9414 demo: office delivery variants

Launch

```
uv run ai9414 demo strips
```

Useful examples:

- canonical_delivery
- robot_starts_mail_room
- keycard_in_office_b
- unlocked_lab

What to inspect:

- Initial state facts.
- Applicable actions.
- The plan trace, one action at a time.
- Compare the solver stats across variants.

Problem variant

In the `unlocked_lab` variant, the lab door starts unlocked.

Which action should disappear from the plan?

- A. `pickup_keycard`
- B. `pickup_parcel`
- C. `unlock_door`
- D. `drop_parcel`

Problem variant

Correct answer: **C**.

If the door is already unlocked, `unlock_door` is no longer applicable.
In the current ai9414 STRIPS example, this also means the
keycard action is no longer useful for reaching `at(parcel, lab)`.

Compare `unlocked_lab` with `canonical_delivery`:
changing one initial fact changes which actions are useful.

Heuristics make forward planning practical

Problem

- Forward search can have a large branching factor.
- Many legal actions do not help the goal.
- BFS memory grows quickly with plan length.

Common ideas

- Relax the problem to estimate distance to the goal.
- Ignore some delete effects.
- Abstract away details.
- Search with a goal-directed heuristic.

Brief detour: Backward regression

Forward progression

$$s \xrightarrow{a} \gamma(s, a)$$

Start with s_0 , apply actions, test the goal.

Backward regression

$$g \xleftarrow{a} \gamma^{-1}(g, a)$$

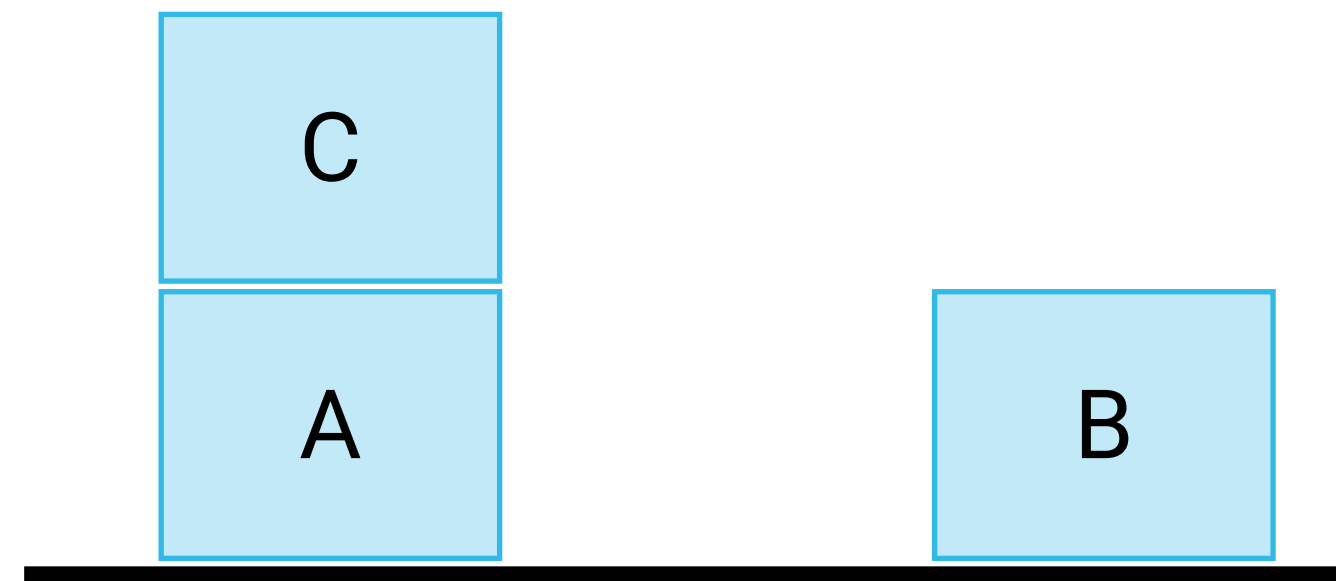
Start with the goal and ask which action could have made it true.

Office example: for $g = \{\text{at}(\text{parcel}, \text{lab})\}$, a relevant last action is $\text{drop_parcel}(\text{robot}, \text{parcel}, \text{lab})$, giving the prior subgoals $\text{at}(\text{robot}, \text{lab})$ and $\text{carrying}(\text{robot}, \text{parcel})$.

Interacting goals: blocks world

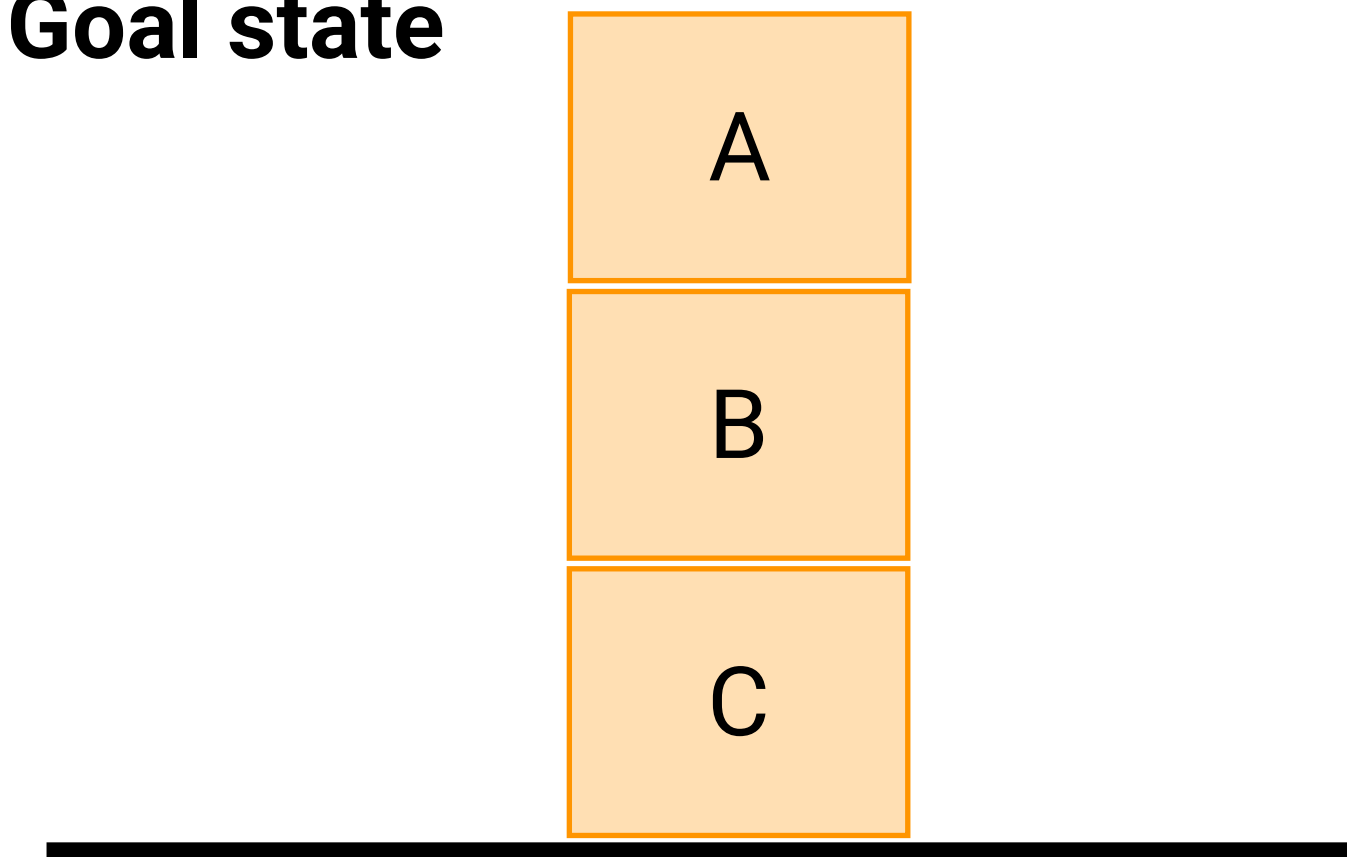
The Sussman anomaly is a tiny blocks-world problem where the goals interact.

Initial state



$\text{on}(C,A), \text{ontable}(A), \text{ontable}(B)$

Goal state



$\text{on}(A,B), \text{on}(B,C)$

Solve one goal first

Suppose we try to achieve $\text{on}(A, B)$ first:

Plan prefix:

1. $\text{unstack}(C, A)$
2. $\text{putdown}(C)$
3. $\text{pickup}(A)$
4. $\text{stack}(A, B)$

Resulting state



$\text{on}(A, B)$, but B is not clear.

Why the subgoal order fails

To put B onto C, the planner needs `clear(B)`.

But after `stack(A, B)`:

- `on(A, B)` is true
- `clear(B)` is false
- the only way to make `clear(B)` true is to remove A.

That means applying:

`unstack(A, B)`

which deletes `on(A, B)`, the goal just achieved.

Interleaving the goals

A short valid plan makes $\text{on}(B, C)$ before $\text{on}(A, B)$.

1.unstack(C,A)

2.putdown(C)

3.pickup(B)

4.stack(B,C)

5.pickup(A)

6.stack(A,B)

The point of this

- Goals are not always independent.
- Delete effects reveal goal interactions.
- A planner should reason over the whole state and whole goal set.

Regression view of the same problem

Backward reasoning helps explain relevance:

For $\text{on}(A, B)$ a relevant last action is: For $\text{on}(B, C)$ a relevant last action is:

$\text{stack}(A, B)$

$\text{stack}(B, C)$

Prior subgoals include:

$\text{holding}(A), \text{clear}(B)$

Prior subgoals include:

$\text{holding}(B), \text{clear}(C)$

The problem is the shared resource: $\text{clear}(B)$ is needed for one goal and destroyed by achieving the other too early.

Limits of classical planning

What can go wrong

- The initial state may be incomplete or wrong.
- An action may have unmodelled preconditions.
- Effects may be nondeterministic.
- Other agents may change the world.

Office example

- The keycard is not really in office A.
- The door reader is offline.
- The parcel is too large for the robot's hand.
- Someone moves the parcel during execution.

The qualification problem

It is usually impossible to list every condition that must hold for an action to work.

For unlock_door

- keycard is valid;
- door has power;
- robot can reach the reader;
- reader software is working;
- no emergency override blocks entry.

Implication

Planning needs a model, but every model is an abstraction.

A plan is only as reliable as the knowledge encoded in the state and action representation.

What knowledge is missing?

The plan says the robot can unlock the lab door. What if, in the real building, the card reader sometimes loses network access.

Which modelling change is most direct?

- A. Add `reader_online(lab_door)` as a precondition of `unlock_door`.
- B. Delete `connected(corridor, lab)` from every state.
- C. Make `pickup_parcel` add `unlocked(lab_door)`.
- D. Change the goal to `at(robot, lab)`.

Missing knowledge

Correct answer: **A**

This is the knowledge representation lesson:

- if a condition is required for an action to work, it belongs in the action model;
- if the planner does not know that condition, it can return a plan that looks valid but fails during execution;
- real systems need monitoring and replanning when model assumptions are violated.

Beyond the clean model

Conditional planning

Include sensing and branches for different observations.

Monitoring

Check whether planned conditions still hold during execution.

Replanning

Repair or replace the plan when reality diverges.

Real systems often combine all three.

Why Explicit Knowledge Representation Is Needed

Planning needs

- objects and types;
- predicates and state variables;
- static domain facts;
- action preconditions and effects;
- invariants and constraints.

Without explicit knowledge

- relevant actions are hard to identify;
- hidden assumptions become bugs;
- impossible goals are hard to explain;
- execution failures are hard to diagnose.

Beyond STRIPS: Task and Motion Planning

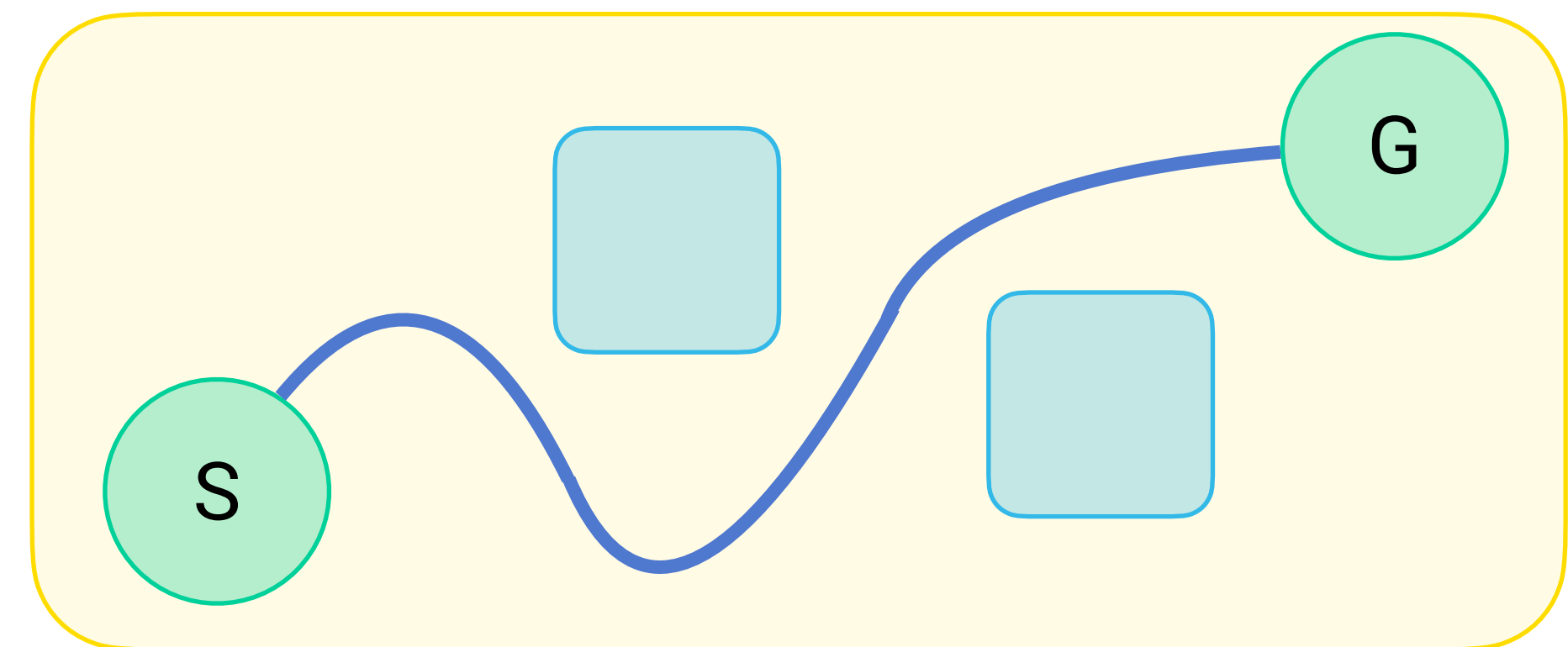
Task planning

- Choose *what to do*
- reason with objects, facts, goals
- produce a sequence such as:

```
move(corridor, mail_room)  
pickup(parcel)  
move(mail_room, lab)
```

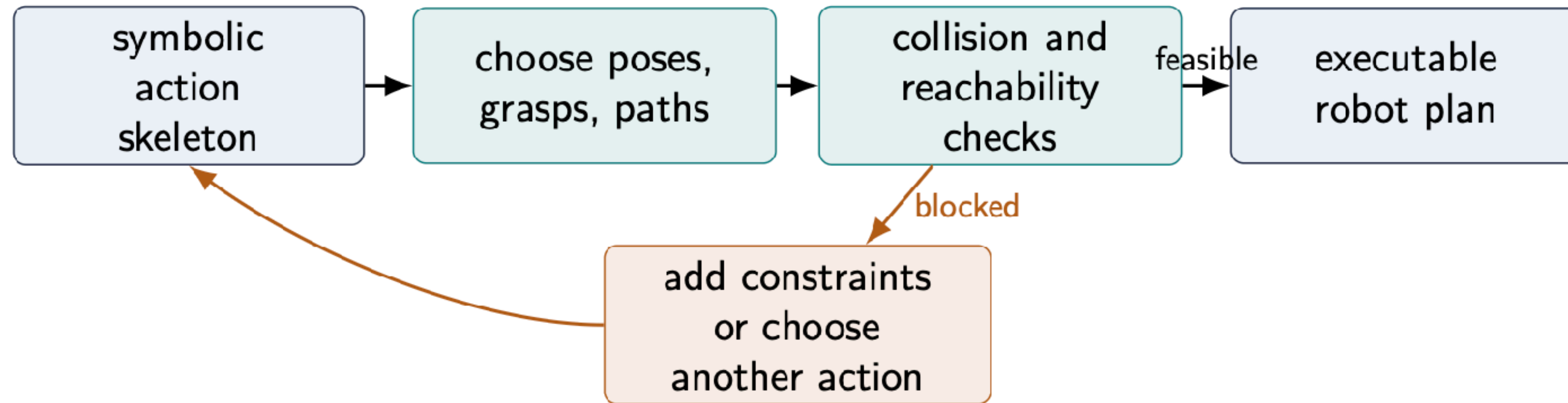
Motion planning

- choose how to move;
- reason with geometry, collisions, reachability;
- produce paths, poses, and controls.



Task and Motion Planning (TAMP) studies the boundary between symbolic choices and physically feasible execution.

TAMP as coupled search



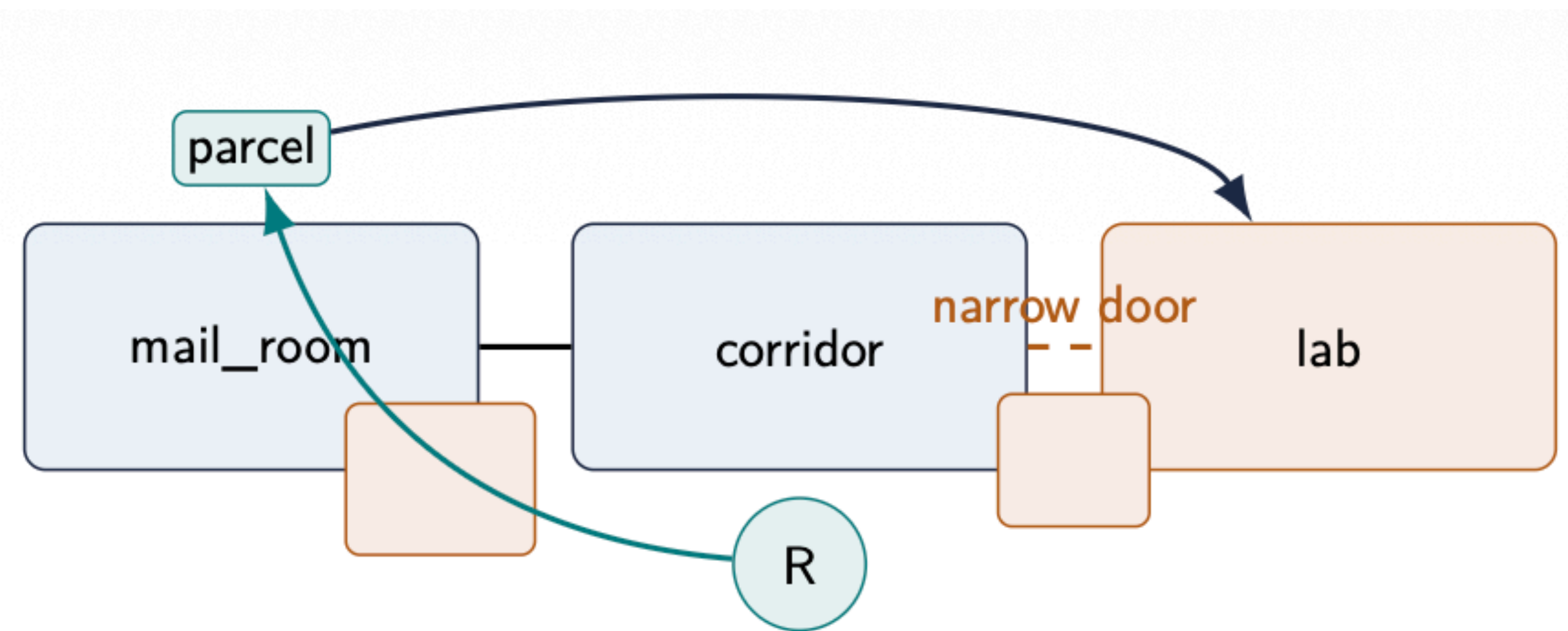
Task level tasks

- Which object should move first
- Which room should the robot visit?
- Which symbolic preconditions must become true?

Motion level tasks

- Where can the robot stand?
- Can the arm reach the parcel?
- Is there a collision-free path through the door?

Office Delivery as a TAMP Problem



same symbolic action, many geometric choices

A STRIPS action hides details

`pickup_parcel(robot, parcel, mail_room)`

TAMP must also decide:

- the robot's standing pose;
- the parcel grasp;
- a path to the parcel;
- a path through the door while carrying it;
- whether the geometry makes the plan impossible.

TAMP: Practical but Not General-Purpose

Works well when

- the robot, objects, and rooms are carefully modelled;
- good grasp and pose choices can be generated;
- collision checking is reliable;
- the task is long, but the world is mostly predictable.

Still hard when

- perception gives incomplete or wrong state information;
- clutter, contact, or deformable objects matter;
- the environment changes during execution;
- the planner must scale without losing robustness.

STRIPS hides physical feasibility; TAMP makes feasibility part of the planning problem.

Takeaways

- Planning generalises search by making states, goals, and actions explicit.
- STRIPS-style actions use preconditions plus add/delete effects.
- Forward state-space planning searches from the initial state through generated symbolic states.
- The office robot plan works because the model captures key dependencies: keycard before locked door, parcel before delivery.
- Classical planning is powerful but brittle when the world violates its assumptions.

Sources

Useful sources for these definitions and examples:

- Pittsburgh “Planning and Search / Classical Planning” deck for the main skeleton and add/delete transition rule.
- Dana Nau, Automated Planning: Theory and Practice, Chapter 2 and Chapter 4 slides for formal definitions and state-space planning.
- ALMA Chapter 11 planning slides for canonical STRIPS framing.
- Philipp Koehn’s 2026 planning deck for the search-vs-planning framing and real-world limitations.
- Our ai9414 STRIPS demo for the office delivery robot example.